LAPORAN PRAKTIKUM ALGORITMA DAN PEMROGRAMAN
ARRAY, POINTER DAN STRUKTUR

dibuat untuk memenuhi tugas Mata Kuliah Elektronika Digital

Dosen Pengampu:

Muhamad Yamin, S.T, M.T.



Disusun oleh:

Nama : Winawati

NIM : 231331062

PROGRAM STUDI D3 TEKNIK TELEKOMUNIKASI

JURUSAN TEKNIK ELEKTRO

POLITEKNIK NEGERI BANDUNG

2024

KETERANGAN

1. Judul Praktikum : Array, Pointer dan Struktur
2. Tanggal Praktikum : 12 November 2024
3. Tanggal Pengumpulan Laporan : 14 November 2024
4. Nama Pembuat/Mahasiswa : Winawati
5. Nama Dosen : Muhamad Yamin, S.T, M.T.

DAFTAR ISI

KETERANGAN 1

DAFTAR ISI 2

I. TUJUAN PRAKTIKUM 1

II. LANDASAN TEORI 1

1. Array 1

2. Pointer 2

2.1 Mendeklarasikan Variabel Pointer 2

2.2 Mengatur Pointer agar Menunjuk ke Variabel Lain 3

2.3 Mengakses Isi Suatu Variabel Melalui Pointer 3

2.4 Pointer dan Array 3

3. Struktur 4

3.1 Array dan Struktur 4

III. ALAT-ALAT YANG DIGUNAKAN PRAKTIKUM 5

IV. ANALISIS PRAKTIKUM **Error! Bookmark not defined.**

I. TUJUAN PRAKTIKUM

Tujuan dari Percobaan Praktikum ini mahasiswa diharapkan mampu menguasai beberapa aspek penting yang menjadi tujuan percobaan praktikum kali ini. Dengan tujuan praktikum sebagai berikut:

1. Mahasiswa dapat memahami dan mengidentifikasi struktur data array dalam bahasa C.
2. Mahasiswa dapat memahami konsep pointer dalam bahasa C.
3. Mahasiswa dapat memahami konsep struktur dalam bahasa C.
4. Mahasiswa dapat mengerti perbedaan penggunaan array dan pointer.
5. Mahasiswa dapat mengerti perbedaan array dan struktur.

II. LANDASAN TEORI

1. Array

Suatu array berdimensi satu dideklarasikan dalam bentuk umum berupa :

```
tipe_data nama_var[ukuran];
```

Dengan:

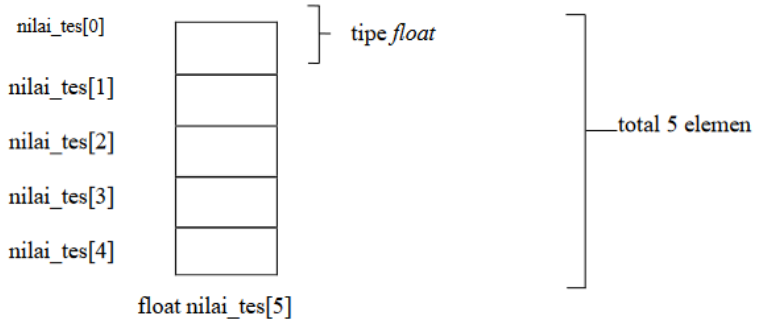
- tipe_data : untuk menyatakan tipe dari elemen array, misalnya int, char, float.
- nama_var : nama variabel array.
- ukuran : untuk menyatakan jumlah maksimal elemen array.

Contoh pendeklarasian array :

```
float nilai_tes[5];
```

menyatakan bahwa array nilai_tes mengandung 5 elemen bertipe float.

Pada C, data array akan disimpan dalam memori yang berurutan. Elemen pertama mempunyai indeks bernilai 0. Jika variabel nilai_tes dideklarasikan sebagai array dengan 5 elemen, maka elemen pertama memiliki indeks sama dengan 0, dan elemen terakhir memiliki indeks 4. Gambar 1.1 di bawah ini menjelaskan urutan komponen dalam array.



Gambar 1.1 Array berdimensi satu

Bentuk umum pengaksesan array adalah sbb :

```
nama_var[indek
```

Sehingga, untuk array nilai_tes, maka :

`nilai_tes[0]` → elemen pertama dari **nilai_tes**

`nilai_tes[4]` → elemen ke-5 dari **nilai_tes**

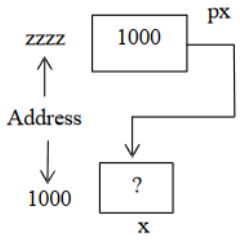
Contoh :

```
nilai_tes[0] = 70;                    /* contoh 1 */
scanf("%f", &nilai_tes[2]);           /* contoh 2 */
```

Contoh pertama merupakan pemberian nilai 70 ke nilai_tes[0]. Sedangkan contoh 2 merupakan perintah untuk membaca data bilangan dari keyboard dan diberikan ke nilai_tes[2].

2. Pointer

Variabel pointer sering dikatakan sebagai variabel yang menunjuk ke obyek lain. Pada kenyataan yang sebenarnya, variabel pointer berisi alamat dari suatu obyek lain (yaitu obyek yang dikatakan ditunjuk oleh pointer). Sebagai contoh, px adalah variable pointer dan x adalah variabel yang ditunjuk oleh px. Kalau x berada pada Alamat memori (alamat awal) 1000, maka px akan berisi 1000. Sebagaimana diilustrasikan pada gambar 8.1 di bawah ini



Gambar 1.2 Variabel pointer px menunjuk ke variabel x

2.1 Mendeklarasikan Variabel Pointer

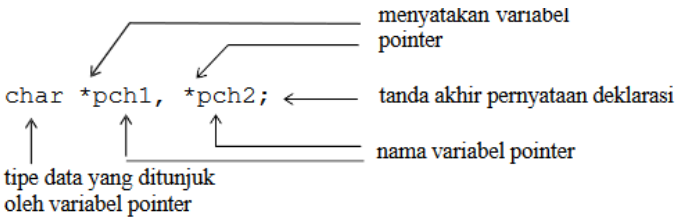
Suatu variabel pointer dideklarasikan dengan bentuk sebagai berikut :

```
tipe *nama_variabel
```

dengan tipe dapat berupa sembarang tipe data dalam bahasa C. Adapun nama_variabel adalah nama dari variabel pointer. Sebagai contoh :

```
int *px;                / *contoh 1 */
char *pch1, *pch2;      / *contoh 2 */
```

Contoh pertama menyatakan bahwa px adalah variabel pointer yang menunjuk ke suatu data bertipe int, sedangkan contoh kedua masing pch1 dan pch2 adalah variabel pointer yang menunjuk ke data bertipe char.



Gambar 8.2 Ilustrasi pendeklarasian variabel pointer

2.2 Mengatur Pointer agar Menunjuk ke Variabel Lain

Agar suatu pointer menunjuk ke variabel lain, mula-mula pointer harus diisi dengan alamat dari variabel yang akan ditunjuk. Untuk menyatakan alamat dari suatu variabel, operator & (operator alamat, bersifat unary) bisa dipergunakan, dengan menempatkannya di depan nama variabel. Sebagai contoh, bila x dideklarasikan sebagai variabel bertipe int, maka

&x

berarti "alamat dari variabel x". Adapun contoh pemberian alamat x ke suatu variable pointer px (yang dideklarasikan sebagai pointer yang menunjuk ke data bertipe int) yaitu :

px = &x;

Pernyataan di atas berarti bahwa px diberi nilai berupa alamat dari variabel x. Setelah pernyataan tersebut dieksekusi barulah dapat dikatakan bahwa px menunjuk ke variable x.

2.3 Mengakses Isi Suatu Variabel Melalui Pointer

Jika suatu variabel sudah ditunjuk oleh pointer, variabel yang ditunjuk oleh pointer tersebut dapat diakses melalui variabel itu sendiri (pengaksesan langsung) ataupun melalui pointer (pengaksesan tak langsung). Pengaksesan tak langsung dilakukan dengan menggunakan operator indirection (tak langsung) berupa simbol * (bersifat unary). Contoh penerapan operator * yaitu :

*px

yang menyatakan "isi atau nilai variabel/data yang ditunjuk oleh pointer px" . Sebagai contoh jika y bertipe int, maka sesudah dua pernyataan berikut

px = &x;

y = *px;

y akan berisi nilai yang sama dengan nilai x.

2.4 Pointer dan Array

Hubungan antara pointer dan array pada C sangatlah erat. Sebab sesungguhnya array secara internal akan diterjemahkan dalam bentuk pointer. Pembahasan berikut akan memberikan gambaran hubungan antara pointer dan array. Misalnya dideklarasikan di dalam suatu fungsi

```
static int tgl_lahir[3] = { 01, 09, 64 };
```

dan

```
int *ptgl;
```

Kemudian diberikan instruksi

```
ptgl = &tgl_lahir[0];
```

maka ptgl akan berisi alamat dari elemen array tgl_lahir yang berindeks nol. Instruksi di atas bisa juga ditulis menjadi

```
ptgl = tgl_lahir;
```

sebab nama array tanpa tanda kurung menyatakan alamat awal dari array. Sesudah penugasan seperti di atas,

```
*ptgl
```

dengan sendirinya menyatakan elemen pertama (berindeks sama dengan nol) dari array tgl_lahir.

3. Struktur

Struktur adalah koleksi dari variabel yang dinyatakan dengan sebuah nama, dengan sifat setiap variabel dapat memiliki tipe yang berlainan. Struktur dapat digunakan untuk mengelompokkan beberapa informasi yang berkaitan menjadi sebuah satu kesatuan.

Bentuk umum deklarasi struktur adalah sebagai berikut:

```
struct nama_tipe_struktur
{
      tipe field1;
      .
      .
      tipe fieldN;
} variabel_struktur1, …, variabel_strukturM;
```

Elemen dari struktur dapat diakses dengan menggunakan bentuk:

```
variabel_struktur.nama_field
```

Bentuk umum deklarasi struktur adalah sebagai berikut:

```
variabel_struktur.nama_field
```

3.1 Array dan Struktur

Elemen-elemen dari suatu array juga dapat berbentuk sebuah struktur. Misalnya array yang dipakai untuk menyimpan sejumlah data siswa (struct student). Array struktur berdimensi satu ini membentuk suatu tabel, dengan barisnya menunjukkan elemen dari array-nya dan kolomnya menunjukkan elemen dari struktur.

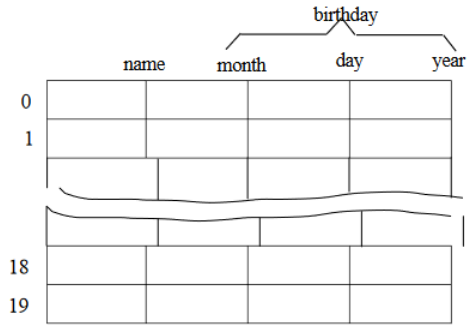
Dalam hal ini maka deklarasi yang dibutuhkan adalah sebagai berikut :

```
#define MAKS 20
     .
     .
     .
struct date {          /* definisi dari tipe date
*/
     int month;
     int day;
     int year;
};
```

```
struct person {  /* definisi dari tipe person */
     char name[30];
     struct date birthday;
};

/* deklarasi dari variabel array student */
struct person student[MAKS];
```

yang artinya, mendeklarasikan array student yang memiliki elemen yang bertipe struct person sebanyak MAKS. Setelah array student dideklarasikan, maka ruang yang disediakan ditunjukkan dalam gambar 9.4 di bawah ini.



Gambar 9.4 Array dari struktur

Elemen-elemen dari array stuktur tersebut bisa diakses dengan cara sebagai berikut :

```
for (i=0; i<MAKS; i++)
{
    printf("Name                : ");
    fgets(student[i].name, sizeof student[i].name,
        stdin);
    printf("Birthday (mm-dd-yyyy): ");
    scanf("%d-%d-%d", &student[i].birthday.month,
        &student[i].birthday.day,
        &student[i].birthday.year);
    printf("\n");

    /* hapus sisa data dalam penampung keyboard */
    fflush(stdin);
};
```

III. ALAT-ALAT YANG DIGUNAKAN PRAKTIKUM

Alat-alat yang digunakan dalam praktikum terdiri dari:

1. Laptop
2. Aplikasi Dev C+
3. Aplikasi Visio